

Dizgiler: Yapılar ve Algoritmalar

CEN207 Veri Yapıları — Hafta 12

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	C dizgileri, arabellekler, trie'ler, sıkıştırılmış trie'ler, sonek dizileri
2	Saf arama, KMP (başarısızlık işlevi + arama), Rabin-Karp
3	Boyer-Moore, Z algoritması, dinamik programlama: düzenleme uzaklığı, LCS

Öğrenme çıktıları: ÖÇ.1 (veri yapılarını açıklama) · ÖÇ.2 (karmaşıklık analizi) · ÖÇ.6 (dinamik programlama) · ÖÇ.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
C dizgileri, arabellekler, trie'ler, radix ağaçları, sonek dizileri	Bölüm 1–5
Saf, KMP, Rabin-Karp, Boyer-Moore, Z algoritması	Bölüm 6–11
Dinamik programlama: düzenleme uzaklığı, LCS	Bölüm 13–14

Kod örnekleri nasıl çalışır

- Her fikrin eksiksiz bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-12/c/` ve `code/week-12/java/`
- Her programın beklenen çıktısı hafta notlarında

Tekrar — diziler ve char dizisi (Hafta 1)

- **Dizi:** bitişik bellek, indeksle $O(1)$ erişim
- Bir C dizgisi yalnız bir `char` dizisidir
- Hafta 1'in indeks aritmetiği (`base + i`) hâlâ geçerli
- Bugün üstüne bir kural ekliyor: `'\0'` sonu işaretler

Tekrar — ağaçlar ve hash'leme (Hafta 4 ve 6)

- **Ağaçlar:** çocukları olan özyinelemeli düğümler (Hafta 4)
- Bir trie düğümü, harf başına bir çocuğu olan bir ağaç düğümüdür
- **Hash'leme:** bir işlem bir konumu $O(1)$ 'de hesaplar (Hafta 6)
- Rabin-Karp hash'lemeyi yeniden kullanır — ama onu **kaydırır**

Yepyeni bir fikir: dinamik programlama

- Hiçbir önceki hafta bir problemi **çakışan** alt problemlere bölmedi
- Her alt problemi **yalnız bir kez** çözün, bir tabloda saklayın
- Bölüm 13–14 bunu temel ilkelerden tanıtıyor
- İki klasik örnek: düzenleme uzaklığı ve LCS

Bu haftanın haritası — bir bakışta

Yapılar	Arama	Dinamik programlama
C dizgileri, arabellekler	Saf, KMP, Rabin-Karp	Düzenleme uzaklığı
Trie'ler, radix ağaçları	Boyer-Moore, Z algoritması	LCS
Sonek dizileri		

1. C Dizgileri: Bellek, `'\0'`, ve `strlen`

Başlangıç sorusu

Şimdiye kadarki her yapı boyutunu bir yerde sakladı. Bir C dizgisi saklamaz. Öyleyse bir işlem nerede bittiğini nasıl bilir?

Kısa bir tarihçe

- C, Bell Labs'ta, 1970'lerin başında tasarlandı (Dennis Ritchie)
- Uzunluk alanı yok: yalnız bir `char` dizisi + bir kural
- `'\0'` (NUL, değer 0) "dizgi burada bitiyor" der
- Bu tek seçim arabellek taşmasını olanaklı kılar

Sezgi — bir posta kutuları sırası

- Her posta kutusu (bayt) bir harf tutar
- Hiçbir işaret "bu son dolu kutu" demez
- Bunun yerine: ilk **boş** kutu sonu işaretler
- Onun ötesine yürümek, başkasının postasına girmektir

Fikir: dizi + kural

- Bir C dizgisi = bir `char` dizisi, başka hiçbir şey değil
- `'\0'` (NUL) dizgi-sonu işaretidir
- Her işlem onu bulmak için bayt bayt yürür
- Hiçbir kestirme yol yoktur — uzunluk asla önbelleğe alınmaz

Arabellek taşması hatası

- `strcpy` tarzı kopyalama, `'\0'` 'ı da kopyalayana kadar yazar
- Hedef kaynaktan küçükse: sorun
- Son geçerli indisin ötesine yazmak **tanımsız davranıştır**
- Bugün: her yazmayı her zaman bir sınır kontrolüyle **koruruz**

C dizgisi belleği, adım adım

CAP = 16

kaynak metin: "HELLOWORLD" (10 harf)



Bitti: 16 baytlık arabelleğe 10 harf + sonlandırıcı güvenle sığıdı (5 bayt boşta kaldı). strlen her çağrıldığında dizgiyi O(n) baştan tarar.

Uç durum — taşma, bayraklanır, hiç çalıştırılmaz

CAP = 16

kaynak metin: "HELLOWORLD" (10 harf)



Bitti: 16 baytlık arabelleğe 10 harf + sonlandırıcı güvenle sığıdı (5 bayt boşta kaldı). strlen her çağrıldığında dizgiyi O(n) baştan tarar.

Kod — korumalı kopyalama döngüsü

```
int i = 0;
while (src[i] != '\0') {
    if (i == cap) break;
    buf[i] = src[i];
    i++;
}
int overflow = (src[i] != '\0');
if (!overflow) buf[i] = '\0';
```

Kod — strlen onu ikinci kez yürür

```
size_t len = 0;  
if (!overflow)  
    while (buf[len] != '\0') len++;
```

- Hiçbir uzunluk hiçbir yerde önbelleğe alınmaz
- `strlen` 'i bir döngü koşulunda çağırmak: $O(n)$, $O(n^2)$ olur

Karmaşıklık

- L uzunluğunda bir dizgi kopyalamak: $O(L)$
- `strlen` : $O(L)$ — **her tek çağrıda**, sıfırdan yeniden hesaplanır
- Hiçbir önbellekleme, hiçbir kestirme, asla
- Önbelleğe alınmış bir uzunluk, dizgi değiştiği an bozulurdu

Sık yapılan hatalar

- Bir arabelleği boyutlandırırken sonlandırıcı için `+1` 'i unutmak
- `strlen` 'i bir döngünün koşulunun içinde çağırmak
- Dizgileri `strcmp` yerine `==` ile karşılaştırmak

Mini soru

10 karakterlik bir sözcük hangi boyutta
bir arabellek gerektirir? `char buf[10]`
neden bir bayt küçük?

Cevap

11 bayt. On karakter artı `'\0'`
için bir bayt — sonlandırıcıyı unutmak
bu bölümün klasik hatasıdır.

2. Büyüyen Bir Dizgi Arabelleği

Başlangıç sorusu

Bölüm 1'in arabelleği, bir karakter bile yazılmadan önce seçilen sabit bir kapasiteye sahipti. Ya kendini büyütebilseydi?

Sezgi — bir çekmeceyi aşmak

- Bir çekmece sabit sayıda dosya tutar
- Dolu çekmece: **daha büyük** bir tane al, her dosyayı taşı
- Her seferinde "bir boy daha büyük" almak: sürekli taşıma
- **İki katı** boyut almak: taşıma seyrekleşir

Fikir: dolduğunda ikiye katla

- `len` (kullanılan), `cap` (ayrılan), ve depolamayı izleyin
- `len == cap` : **iki katı** boyutta bir blok ayırın
- Var olan her karakteri kopyalayın, sonra yeni yazın
- Katlama, `n` büyüdükçe büyümeyi **üstel olarak seyrekleştirir**

Büyüyen arabellek, adım adım

cap = 16

len = 10



Bitti: 10 harf eklendi, 2 büyüme oldu, son kapasite = 16 (6 bayt boşta). Büyüme ender ve pahalı ($O(\text{len})$) ama toplamda amorti edilmiş $O(1)$ ekleme sağlar.

Uç durum — en küçük olası başlangıç

cap = 16

len = 10



Bitti: 10 harf eklendi, 2 büyüme oldu, son kapasite = 16 (6 bayt boşta). Büyüme ender ve pahalı ($O(\text{len})$) ama toplamda amorti edilmiş $O(1)$ ekleme sağlar.

Kod — büyüt, sonra yaz

```
void append(char c) {  
    if (len == cap) {  
        cap = cap * 2;  
        buf = realloc(buf, cap);  
    }  
    buf[len] = c;  
    len++;  
}
```

Karmaşıklık

- Bir ekleme: genelde $O(1)$, bir büyümede $O(1 \cdot n)$
- n ekleme toplam: $O(n)$ — ekleme başına **amorti edilmiş $O(1)$**
- Büyüme kopyalaması geometrik bir seri oluşturur, n ile sınırlı
- Hafta 1'in dinamik dizisiyle aynı garanti

Sık yapılan hatalar

- Katlama yerine sabit bir miktarla büyütmek: toplam $O(n^2)$
- `realloc` 'tan sonra işaretçiyi güncellemeyi unutmak (C)
- Bir C dizgisi de `+1` gerektirdiğinde `len` bayt ayırmak

Mini soru

Katlama neden amorti edilmiş $O(1)$ verir, ama sabit 10'luk bir büyüme ortalama ekleme başına $O(n)$ verir?

Cevap

Katlama: n/k , $\log n$ büyümeye
küçülür. Sabit $+10$: $n/10$ büyüme,
her biri n 'e kadar kopyalar.

3. Trie'ler: Her Karakter İçin Bir Kenar

Başlangıç sorusu

Bir hash tablosu "X bir sözcük mü?"yü $O(1)$ 'de yanıtlar. "X ile hangi sözcükler başlıyor?"u hiç yanıtlayamaz. Peki ne yanıtlar?

Kısa bir tarihçe

- Edward Fredkin, "Trie Memory," 1960
- Ad "retrieval"dan gelir
- "Tree" ile karışmaması için genellikle "try" diye okunur
- Bugün hâlâ otomatik tamamlamanın arkasındaki standart yapı

Sezgi — bir yol tabelası ağacı

- Yoldaki her çatal tek bir harfle etiketlenmiş
- Bir sözcüğün harflerini, bir çatal bir çatal izleyin
- Paylaşılan önekler aynı erken çatalları paylaşır
- Küçük bir bayrak "tam bir sözcük burada bitiyor"u işaretler

Fikir: paylaşılan önekler düğümleri paylaşır

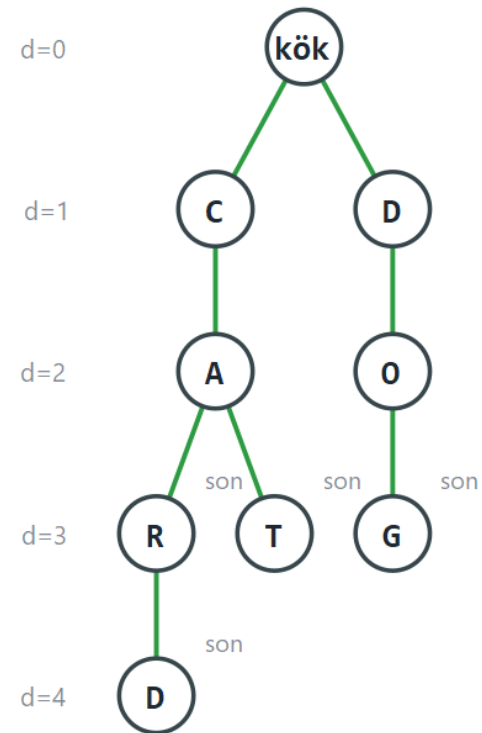
- `insert("CAT")` , sonra `insert("CAR")` : `C` → `A` 'yı paylaşır
- `insert("CARD")` : `C` → `A` → `R` 'ı `"CAR"` ile paylaşır
- `insert("DOG")` : hiçbir şey paylaşmaz, kökten yeni dal
- Bir düğüm **hem** "CAR'ın sonu" **hem de** "CARD'a giden yol" olabilir

Trie işlemleri

İşlem	Maliyet
<code>insert(word)</code>	$O(L)$, L = sözcük uzunluğu
<code>search(word)</code>	$O(L)$
önek kontrolü	$O(L)$

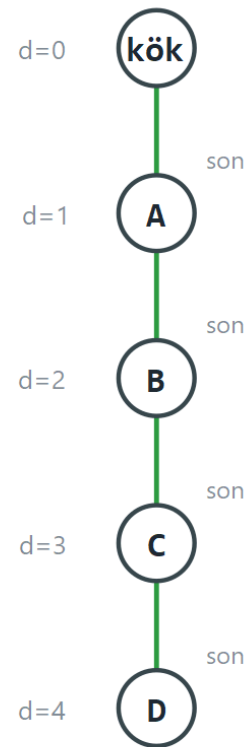
Başka kaç sözcük saklandığından bağımsız!

Trie ekleme ve arama, adım adım



Bitti: 4 sözcük eklendi, 3 arama yapıldı (1 bulundu). Her işlem, sözcüğün uzunluğu L ile $O(L)$ -- trie boyutundan (kaç sözcük olduğundan) bağımsız.

Uç durum — dallanmasız bir zincir



Bitti: 4 sözcük eklendi, 3 arama yapıldı (2 bulundu). Her işlem, sözcüğün uzunluğu L ile $O(L)$ -- trie boyutundan (kaç sözcük olduğundan) bağımsız.

Kod — ekleme

```
void insert(TrieNode *root, const char *word) {
    TrieNode *cur = root;
    for (int i = 0; word[i]; i++) {
        int c = word[i] - 'A';
        if (cur->child[c] == NULL)
            cur->child[c] = new_node();
        cur = cur->child[c];
    }
    cur->isEnd = true;
}
```

Kod — arama

```
bool search(TrieNode *root, const char *word) {
    TrieNode *cur = root;
    for (int i = 0; word[i]; i++) {
        int c = word[i] - 'A';
        if (cur->child[c] == NULL) return false;
        cur = cur->child[c];
    }
    return cur->isEnd;
}
```

Karmaşıklık

- Her işlem: $O(L)$, L = sözcüğün uzunluğu
- Trie'nin toplam sözcük sayısı n 'den bağımsız
- Sıralı bir diziyi ya da dengeli bir ağacı geçer: $O(L \log n)$
- Bedel: bellek — çoğu boş çocuk dizisi

Sık yapılan hatalar

- "Bulundu"yu "bir önektir" ile karıştırmak
- Bir düğümün hem `isEnd` HEM de çocuklu olabileceğini unutmak
- Trie'yi C'de hiç serbest bırakmamak (bir düğüm bir düğüm sızıntı)

Mini soru

Bir trie "DOG" tutuyor. `search("DO")` çağırıyorsunuz. True mü false mü döndürür — ve neden?

Cevap

False. $D \rightarrow 0$ yolu var, böylece
"D0" geçerli bir **önektir** — ama
0 düğümü **isEnd** olarak işaretli değil.

4. Sıkıştırılmış Trie'ler (Radix Ağaçları)

Başlangıç sorusu

"INTERNATIONAL"i (13 harf) düz bir trie'ye eklemek, her biri tek çocuklu 13 yeni düğüm yapar. Daha iyisi olur mu?

Kısa bir tarihçe

- Donald R. Morrison, "PATRICIA," 1968
- Ad: pratik bir erişim algoritması için bir kısaltma
- Genel kullanımda **radix ağacı** da denir
- Aynı fikir gerçek dünya IP yönlendirme tablolarının altında yatar

Fikir: uzat, oluştur, ya da böl

- Sözcük bir kenar etiketiyle tam eşleşir: **in**
- Sıradaki harfle başlayan hiçbir kenar yok: **yeni yaprak kenar**
- Sözcük bir kenar etiketinin yalnız bir kısmını paylaşır: **böl**
- Bir bölme, uyuşmazlıkta yeni bir dallanma düğümü oluşturur

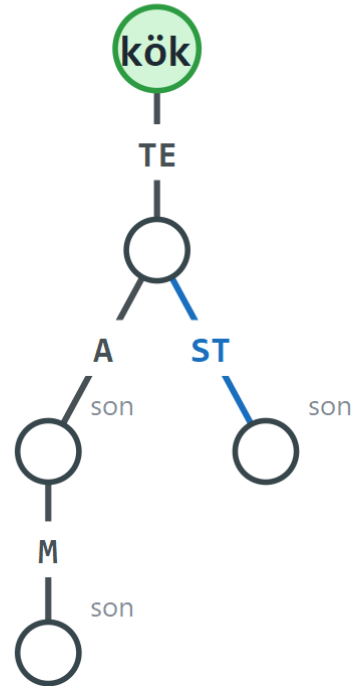
Bir kenarı bölmek

- Yalnız "TEST" : "TEST" etiketli tek bir yaprak kenar
- "TEA" yı ekleyin: ayrılmadan önce yalnız "TE" yi paylaşır
- "TEST" kenarı "TE" de bölünür
- Şimdi iki çocuk: "ST" (eski) ve "A" (yeni)

Bağlantı — Hafta 4'ün Huffman kodlaması

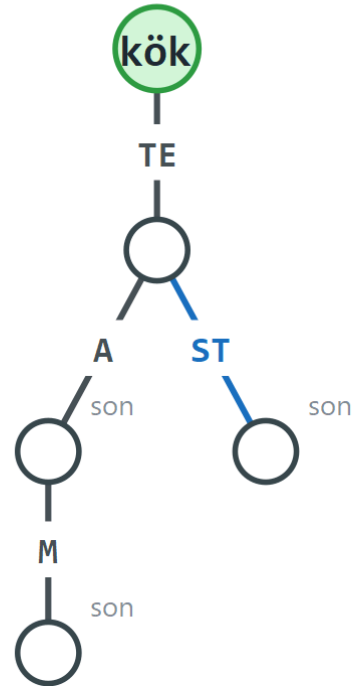
- Huffman: çarpık **sıklıkları** sömürerek sıkıştırır
- Sıkıştırılmış trie: **yapısal** israfı kaldırarak sıkıştırır
- Huffman şekli simgelerin NE SIKLIKLA geçtiğine bağlıdır
- Trie şekli yalnız saklanan gerçek karakterlere bağlıdır

Sıkıştırılmış trie, adım adım



Bitti: 3 sözcük eklendi, 3 arama (1 bulundu). Bölünmeler sayesinde düğüm sayısı sözcüklerin toplam harf sayısından çok daha az kalır.

Uç durum — iç içe bölünmeler



Bitti: 3 sözcük eklendi, 3 arama (1 bulundu). Bölünmeler sayesinde düğüm sayısı sözcüklerin toplam harf sayısından çok daha az kalır.

Kod — ekleme (uzat, oluştur, ya da böl)

```
int j = common_prefix_len(word + i, child->label);
if (j == strlen(child->label)) {
    node = child; i += j;    /* in */
} else {
    RNode *mid = split_edge(node, child, j);
    /* kalan sonek için yeni yaprak */
}
```

Karmaşıklık

- `insert` , `search` : hâlâ $O(L)$, tam olarak düz bir trie gibi
- Tasarruflar **sürede** değil **alanda**
- Düğüm sayısı dallanma noktaları + sözcük sonlarıyla sınırlı
- Toplam karakter sayısı ile değil

Sık yapılan hatalar

- Yanlış uzunlukta bölmek (ortak önek uzunluğu olmalı)
- Bölünen çocuğu yeni ilk harfi altında yeniden anahtarlamaıı unutmak
- "Yol var"ı "sözcük bulundu" saymak (bölüm 3'teki aynı tuzak)

Mini soru

Sıkıştırılmış bir trie yalnız "APPLE" ve "BANANA" tutuyor. Kaç kök-olmayan düğümü var, ve neden bu kadar az?

Cevap

İki. Hiç paylaşılan önek yok,
böylece her sözcük kökten doğrudan
tek bir yaprak kenar olur.

5. Sonek Dizileri

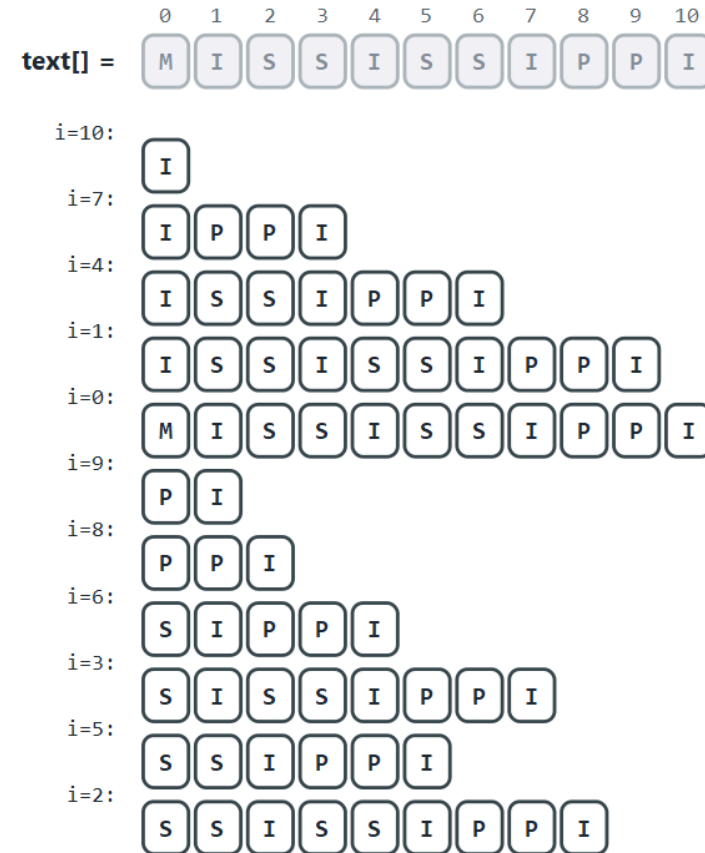
Başlangıç sorusu

Bir trie "X saklı bir sözcük mü?"yü
yanıtlar. Peki ya "P, uzun bir T
metninin herhangi bir yerinde geçiyor mu?"

Fikir: her sonek, sıralı

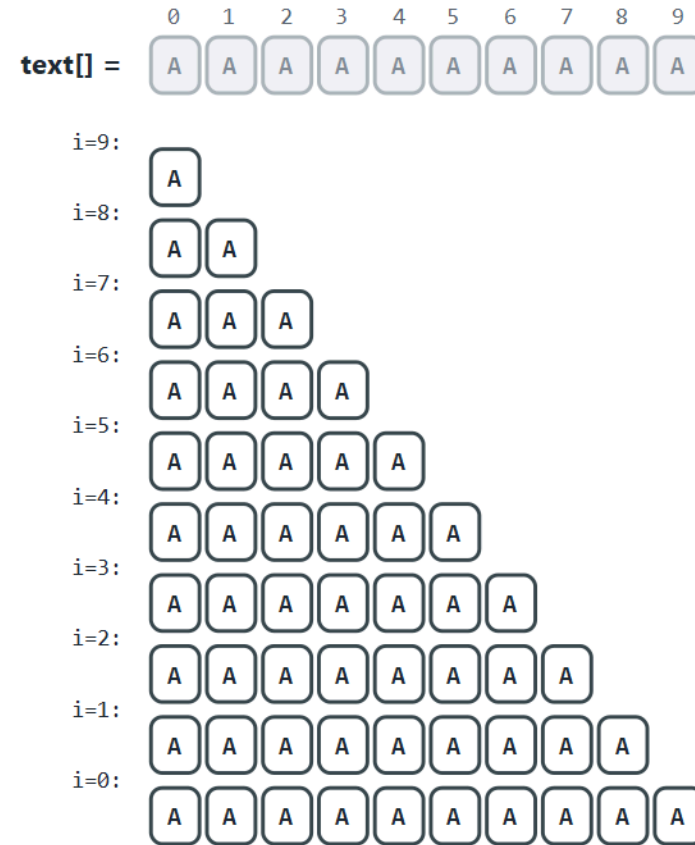
- T'nin **her** soneğini, konum konum listeleyin
- O **n** soneği sözlük sırasına göre sıralayın
- Bir örüntü araması bir **ikili aramaya** döner: $O(m \log n)$
- Hiç bitiş işareti gerekmez — iki sonek her zaman uzunlukta farklı

Sonek dizisi, adım adım



Bitti: sonek dizisi = [10, 7, 4, 1, 0, 9, 8, 6, 3, 5, 2]. Artık pattern aramak, sıralı listede ikili arama ile yapılabilir -- $O(m \log n)$.

Uç durum — her karakter aynı



Bitti: sonek dizisi = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]. Artık pattern aramak, sıralı listede ikili arama ile yapılabilir -- $O(m \log n)$.

Kod — sonekler üzerinde ekleme sıralaması

```
int compare_suffix(const char *text, int a, int b) {  
    return strcmp(text + a, text + b);  
}
```

- Her sonek aynı arabelleğe bir **işaretçidir**
- Sıralama sırasında hiçbir karakter kopyalanmaz

Karmaşıklık

- Buradaki ekleme sıralaması: en kötü durumda $O(n^2)$ karşılaştırma
- Her karşılaştırma: $O(n)$ karaktere kadar
- Gerçek kütüphaneler $O(n \log n)$ ya da $O(n)$ 'de kurar
- Bir kez kurulduktan sonra: örüntü araması $O(m \log n)$

Sık yapılan hatalar

- "Ne olur ne olmaz" diye gereksiz bir bitiş işareti eklemek
- Alt-dizgileri kopyalayarak sonekleri karşılaştırmak (belleği israf eder)
- Dizinin indislerini soneklerin karakterleriyle karıştırmak

Mini soru

Aynı metnin iki soneği neden asla
bayt bayt özdeş olamaz?

Cevap

Her zaman farklı uzunluklara sahiptirler. Sonlu bir dizgide farklı konumlardan başlarlar, böylece uzunluklar farklıdır.

6. Dizgi Eşleştirme Problemi, ve Saf Arama

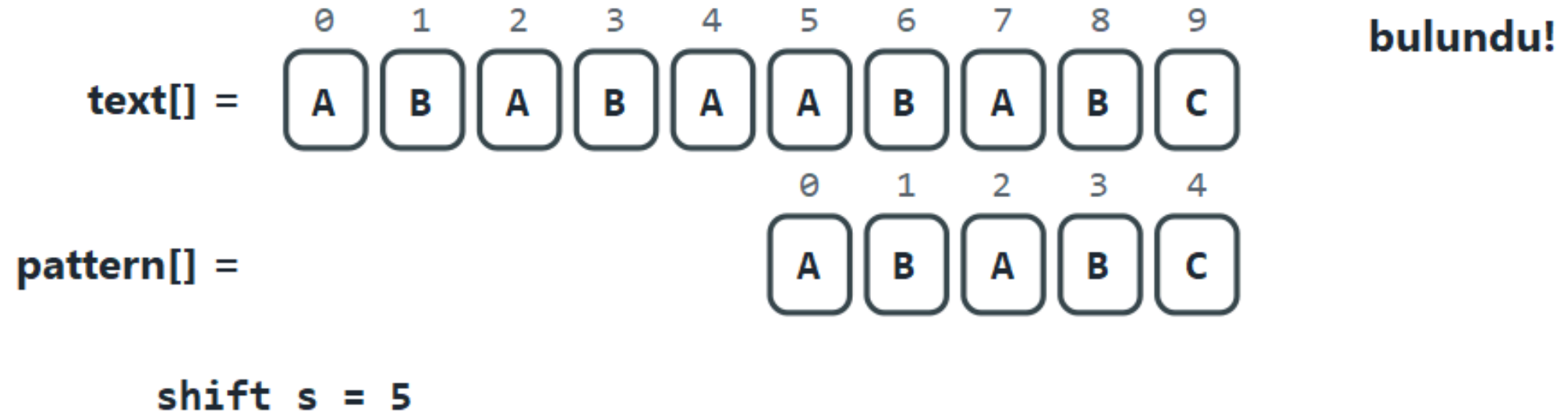
Başlangıç sorusu

Bir metin ve bir örüntü verildiğinde,
örüntü nerede geçer? Bunu yanıtlamanın
en basit olası yolu nedir?

Fikir: her kaydırmayı dene

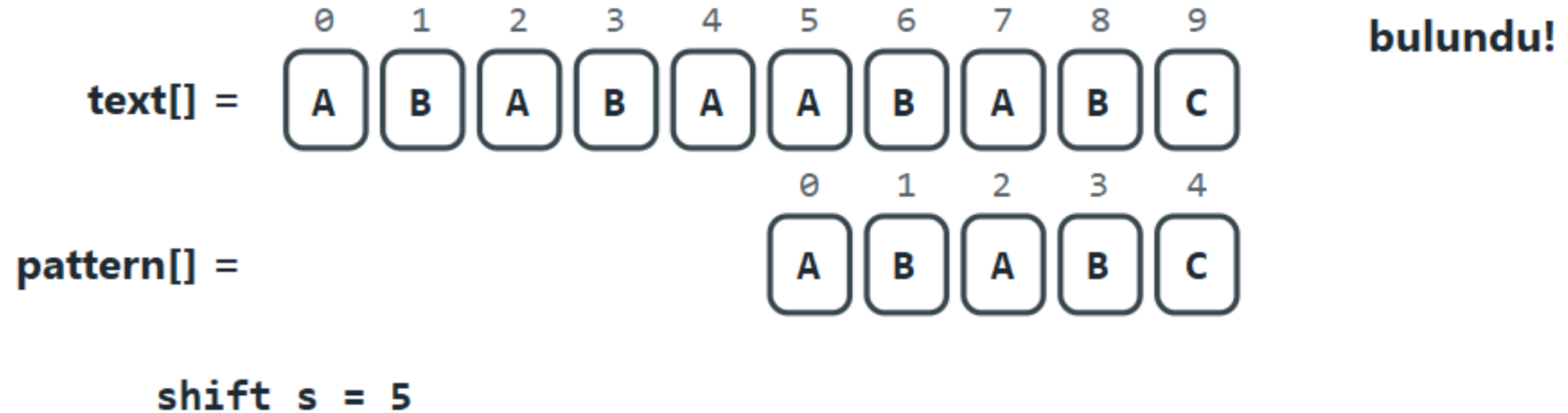
- Örüntüyü metin üzerinde bir seferde bir konum kaydırın
- Her kaydırmada: soldan sağa karşılaştırın
- İlk uyuşmazlıkta durun, ya da tam bir eşleşme kaydedin
- **Bir eşleşmeden sonra devam edin** — oluşumlar çakışabilir

Saf arama, adım adım



Bitti: 6 kaydırma denendi, 1 oluşum bulundu (5). En kötü durumda $O(n*m)$ karşılaştırma.

Uç durum — en kötü durum



Bitti: 6 kaydırma denendi, 1 oluşum bulundu (5). En kötü durumda $O(n*m)$ karşılaştırma.

Kod — çift döngü

```
for (int s = 0; s <= n - m; s++) {  
    int j = 0;  
    while (j < m && text[s+j] == pattern[j])  
        j++;  
    if (j == m) occ[c++] = s;  
}
```

Karmaşıklık

- En iyi durum: $O(n)$ — uyuşmazlıklar hemen olur
- En kötü durum: $O(n*m)$
- "AAAA...AB" vs "AAAB" : neredeyse her kaydırma, neredeyse tam karşılaştırma
- Bu en kötü durum tam olarak Bölüm 7–11'in düzelttiği şey

Sık yapılan hatalar

- Eşleşmelerin çakışabileceğini unutmak (bir isabetten sonra m atlamak)
- s 'yi n - m yerine n 'e kadar döndürmek
- Saf aramanın her zaman "kötü" olduğunu varsaymak — sık kısa girdi için kazanır

Mini soru

Her karşılaştırmayı örüntünün SON karakterine ulaşmaya zorlayan, 2 harflik bir metin ve örüntü oluşturun.

Cevap

`text="AAAAAAAAAA", pattern="AAAB" .`

İlk 3 harf her zaman eşleşir; yalnız son, `'B'`, hiç başarısız olur.

7. Knuth-Morris-Pratt: Başarısızlık İşlevi

Başlangıç sorusu

Saf arama, bir uyuşmazlıktan sonra
öğrendiği her şeyi atar. Ya örüntünün
kendi yapısı bize daha fazlasını söyleseydi?

Kısa bir tarihçe

- Knuth, Morris, ve Pratt, 1977 (SIAM J. Computing)
- Morris & Pratt tarafından, ~1970, bağımsız olarak geliştirildi
- Tablo: `lps[]` — "en uzun uygun önek, aynı zamanda sonek de olan"
- Örüntünün **kendisiyle** karşılaştırılarak kurulur

Fikir: $lps[i]$

- Her `pattern[0..i]` öneki için...
- ...aynı zamanda sonek de olan en uzun uygun önek
- `"ABAB"` : en uzun böyle eşleşme `"AB"` — `$lps[3] = 2$`
- Gelecekteki bir aramaya söyler: "bu kadarı yeniden kullanılabilir"

KMP başarısızlık işlevi, adım adım

	0	1	2	3	4	5	6	7	8	9
pattern[] =	A	B	A	B	C	A	B	A	B	A
	0	1	2	3	4	5	6	7	8	9
lps[] =	0	0	1	2	0	1	2	3	4	3

Bitti: $\text{lps[]} = [0, 0, 1, 2, 0, 1, 2, 3, 4, 3]$. Bu tablo $O(m)$ sürede kuruldu ve KMP aramasının metni asla geri sarmamasını sağlayacak.

Uç durum — geri düşüş bir zinciri izler

	0	1	2	3	4	5	6	7	8	9
pattern[] =	A	B	A	B	C	A	B	A	B	A
	0	1	2	3	4	5	6	7	8	9
lps[] =	0	0	1	2	0	1	2	3	4	3

Bitti: $lps[] = [0, 0, 1, 2, 0, 1, 2, 3, 4, 3]$. Bu tablo $O(m)$ sürede kuruldu ve KMP aramasının metni asla geri sarmamasını sağlayacak.

Kod — lps[]'i kurmak

```
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) {
        lps[i++] = ++len;
    } else if (len != 0) {
        len = lps[len - 1];
    } else {
        lps[i++] = 0;
    }
}
```

Karmaşıklık

- Tüm tabloyu kurmak için $O(m)$
- len , arttığı kadar en fazla azalabilir
- Toplam iş $O(m)$ ile sınırlı, asla $O(m^2)$ değil
- Aynı amorti edilmiş argüman Bölüm 8 ve 11'de yeniden kullanılıyor

Sık yapılan hatalar

- Her uyuşmazlıkta `len` 'i 0'a sıfırlamak (#1 KMP hatası)
- Birer kayma: `pattern[len+1]` değil `pattern[len]` 'i karşılaştırmak
- `lps[0] = 0` 'ın hesaplanmayan sabit bir taban durumu olduğunu unutmak

Mini soru

`lps[m-1]` — en son girdi — tüm örüntü hakkında size ne söyler?

Cevap

Örüntünün en uzun "sınırı" — uygun öneki ki aynı zamanda bütün olarak kendi soneği de olsun.

8. Knuth-Morris-Pratt: Arama

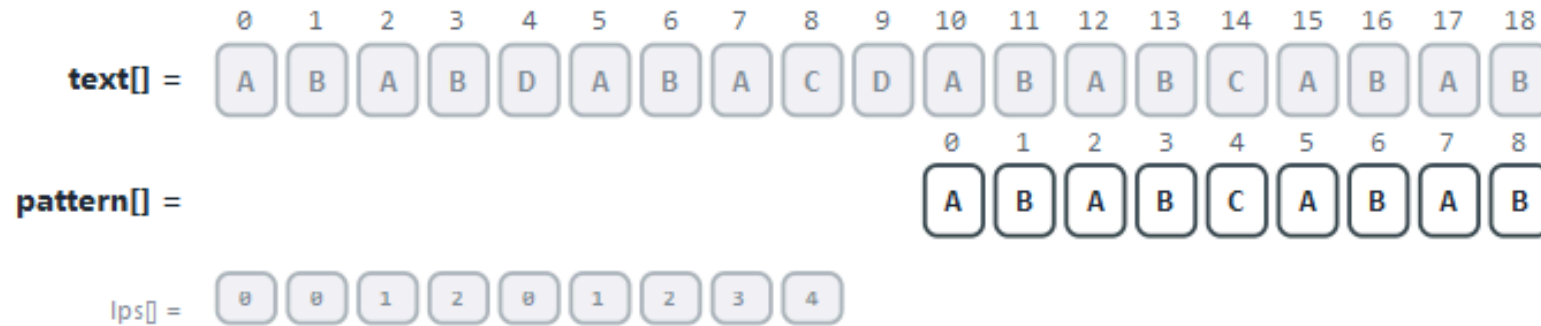
Başlangıç sorusu

`lps[]` elimizdeyken, metni **asla geri gitmeyen** bir işaretçiyle arayabilir miyiz?

Fikir: i asla geri sarmaz

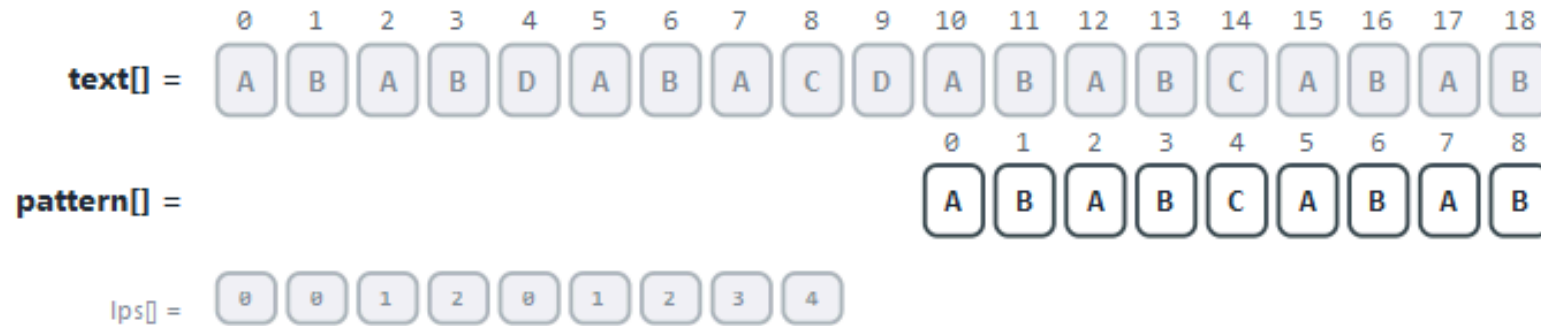
- İki işaretçi: `i` (metin), `j` (örüntü)
- Eşleşme: ikisi de ilerler
- Uyuşmazlık, `j > 0`: `j`, `lps[j-1]`'e geri düşer — `i` yerinde kalır
- Uyuşmazlık, `j == 0`: yalnız `i` ilerler

KMP araması, adım adım



Bitti: i yalnız 19 kez ilerledi (metin uzunluğu kadar), hiç geri gitmedi. 1 oluşum bulundu (10). Toplam süre $O(n + m)$.

Uç durum — çok sayıda lps geri düşüşü



Bitti: i yalnız 19 kez ilerledi (metin uzunluğu kadar), hiç geri gitmedi. 1 oluşum bulundu (10). Toplam süre $O(n + m)$.

Kod — ana döngü

```
while (i < n) {  
    if (text[i] == pattern[j]) {  
        i++; j++;  
        if (j == m) { occ[c++] = i-m; j = lps[j-1]; }  
    } else if (j > 0) {  
        j = lps[j - 1];  
    } else { i++; }  
}
```

Karmaşıklık

- $O(n + m)$: `lps` 'i kurmak için $O(m)$, arama için $O(n)$
- `i` , toplamda, hiçbir zaman `n` 'den fazla ilerlemez
- **Herhangi bir** metin ya da örüntü için geçerli — kötü girdi yok
- Saf aramanın $O(n*m)$ en kötü durumuyla karşılaştırın

Sık yapılan hatalar

- Bir geri düşüşte `i`'yi ilerletmek (tüm garantiyi bozar)
- Bir eşleşmeyi kaydettikten sonra tekrar geri düşmeyi unutmak
- Farklı bir örüntü için kurulmuş bayat bir `lps[]` tablosunu yeniden kullanmak

Mini soru

$O(n+m)$, KMP için neden mümkün ama saf arama için değil, tek bir cümlede?

Cevap

Hiçbir metin karakteri asla yeniden incelenmez — saf aramanın $O(n*m)$ 'si tam olarak onları yeniden incelemekten gelir.

9. Rabin-Karp: Kayan Özet ve Sahte İsabetler

Başlangıç sorusu

Ya karakterleri hiç karşılaştırmak yerine, her pencerenin ucuz bir ÖZETİNİ karşılaştırsaydık?

Kısa bir tarihçe

- Rabin ve Karp, 1987 (IBM J. Research & Development)
- Hafta 6'nın hash'lemesini yeniden kullanır — ama **kaydırır**
- Bir özet eşleşmesi yalnız bir **adaydır**, asla bir kesinlik değil
- Bir eşleşme bildirmeden önce her zaman doğrulanmalı

Fikir: yeniden hesaplama, kaydır

- Her karakteri sabit bir tabanda bir basamak sayın
- Pencereyi kaydırmak: çıkanı çıkar, gireni ekle
- Bir $O(1)$ güncelleme — ortadaki karakterler asla yeniden okunmaz
- Bir özet **eşleşmesi** hâlâ karakter karakter **doğrulanmalıdır**

Rabin-Karp, adım adım

base = 31, mod = 101, $H(\text{pattern}) = 60$

	0	1	2	3	4	5	6	7	8	9
text[] =	H	E	L	L	O	W	O	R	L	D
	0	1	2	3	4					
pattern[] =	W	O	R	L	D					

Bitti: 6 pencere denendi, 1 gerçek eşleşme (5), 0 sahte isabet. Ortalama $O(n + m)$; kötü durumda (çok sahte isabet) $O(n*m)$ 'e yaklaşabilir.

Uç durum — sahte isabetler

base = 4, mod = 7, $H(\text{pattern}) = 3$

text[] =

0	1	2	3	4	5	6	7	8	9	10
A	A	D	B	D	D	B	C	D	B	B

pattern[] =

0	1	2
A	A	D

Bitti: 9 pencere denendi, 1 gerçek eşleşme (0), 2 sahte isabet. Ortalama $O(n + m)$; kötü durumda (çok sahte isabet) $O(n*m)$ 'e yaklaşabilir.

Bu notu hazırlarken yakalanan gerçek bir hata

- İlk C taslağı hash için `long` kullandı, `long long` değil
- Bu dersin araç zincirinde, `long` yalnız **32 bit**
- `mod = 1e9+7` onu taşırdı — sessiz yanlış cevap
- C ve Java çıktısını karşılaştırmak bunu hemen yakaladı

Kod — kaydıran güncelleme

```
if (s > 0)
    tHash = ((tHash - text[s-1]*hPow % mod + mod)
             * base + text[s+m-1]) % mod;
if (tHash == pHash && strcmp(...) == 0)
    occ[c++] = s;    /* HER ZAMAN DOĞRULA */
```

Karmaşıklık

- Ortalama durum: $O(n + m)$ — pencere başına $O(1)$
- Yalnız özeti gerçekten eşleşen pencereler için ekstra $O(m)$
- En kötü durum: modül küçük/kötü seçilmişse $O(n*m)$
- Büyük bir asal modül bu en kötü durumu yok denecek kadar azaltır

Sık yapılan hatalar

- Doğrulamayı atlamak (yalnız hız değil, bir doğruluk hatası)
- "Basitlik için" çok küçük bir modül kullanmak
- Aritmetik için çok dar bir tür kullanmak (yukarıdaki `long` hatası)

Mini soru

Her Rabin-Karp özet eşleşmesi neden hâlâ karakter karakter doğrulanmalıdır?

Cevap

Güvercin yuvası ilkesi — olası
alt-dizgi hash değerinden daha çoksa,
bazıları çakışmak ZORUNDA. Hata değil; matematik.

10. Boyer-Moore: Kötü Karakter Kuralı

Başlangıç sorusu

Şimdiye kadarki her algoritma soldan sağa karşılaştırır. Ya SAĞDAN SOLA karşılaştırmak daha fazla atlamamızı sağlasaydı?

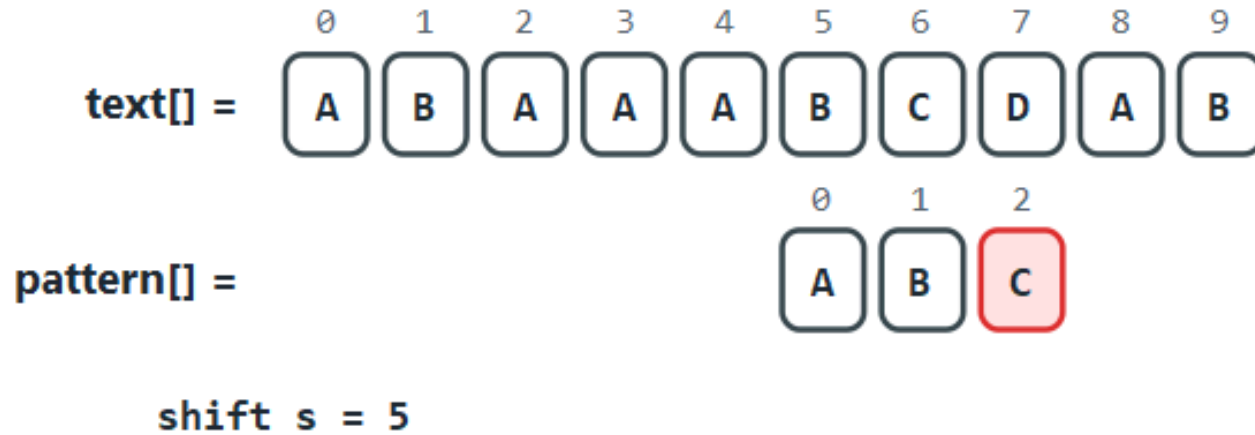
Kısa bir tarihçe

- Boyer ve Moore, 1977 (Communications of the ACM)
- Bu ders yalnız **kötü karakter** kuralını kapsar
- Tam algoritma ikinci bir "iyi sonek" kuralı ekler
- Bugün pratikte sıkça en hızlı dizgi araması

Fikir: geriye tara, gördüğünü kullanarak atla

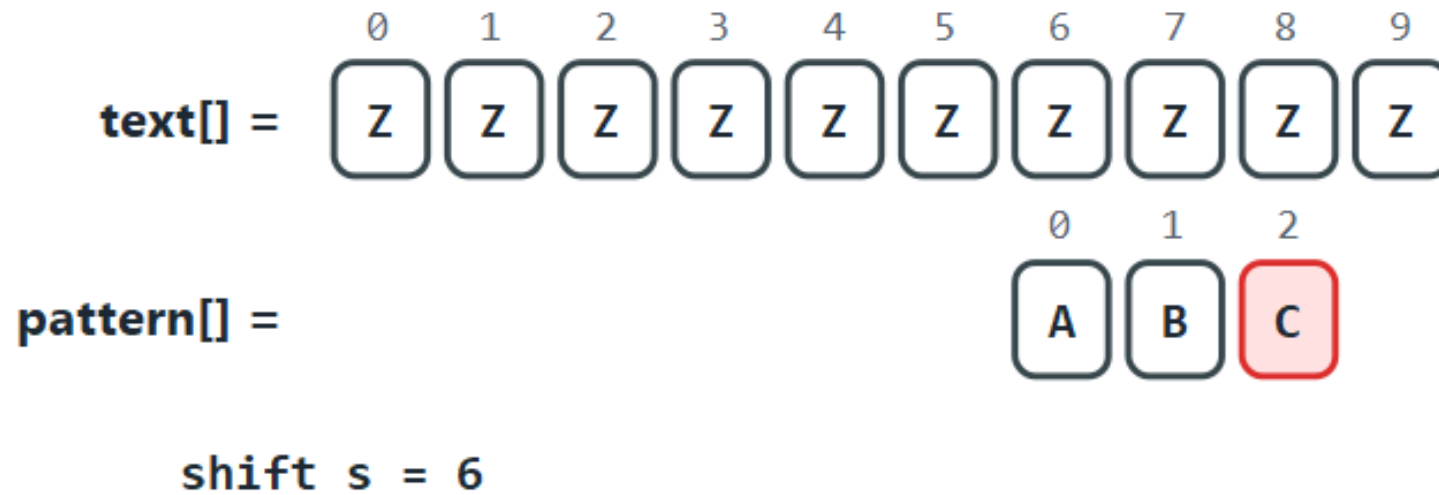
- `pattern[m-1]` 'i önce, sonra `m-2` , ... sağdan sola karşılaştıran
- Uyuşmazlık: o karakterin örüntüdeki SON oluşumuna bakın
- Karakter örüntüde yoksa: TÜM örüntü uzunluğunca atlayın
- Atlama asla 1'den az değil (asla geri, asla yerinde)

Boyer-Moore, adım adım



Bitti: 1 oluşum bulundu (4). Kötü karakter kuralı, geniş alfabede sık büyük atlamalar sağlar -- en iyi durumda $O(n/m)$.

Uç durum — olası en büyük atlama



Bitti: 0 oluşum bulundu ([object Object]). Kötü karakter kuralı, geniş alfabede sık büyük atlamalar sağlar -- en iyi durumda $O(n/m)$.

Kod — kötü karakter tablosu ve atlama

```
for (int c = 0; c < 256; c++) last[c] = -1;
for (int j = 0; j < m; j++) last[pattern[j]] = j;
/* ... */
int shift = j - last[text[s + j]];
s += shift > 1 ? shift : 1;
```

Karmaşıklık

- En iyi durum: $O(n/m)$ — büyük atlamalar, zengin alfabe
- En kötü durum: $O(n*m)$ — düşük çeşitlilikli metin (yalnız kötü karakter kuralı)
- Pratikte doğal dil metninde sıkça en hızlısı
- İyi son ek kuralı (kapsanmadı) en kötü durumu düzeltir

Sık yapılan hatalar

- `shift >= 1` korumasını unutmak (onsuz sonsuza dek döngüye girebilir)
- Kötü karakter tablosunu METİNDEN kurmak, örüntüden değil
- Alışkanlıkla soldan sağa karşılaştırmak — tüm avantajı kaybeder

Mini soru

Boyer-Moore, saf aramanın incelemek zorunda olduğu metin karakterlerini neden atlayabilir?

Cevap

Tek bir karşılaştırma birkaç
kaydırmanın imkansız olduğunu kanıtlar
— orada eşleşme olamaz, hiç kontrol edilmez.

11. Z Algoritması

Başlangıç sorusu

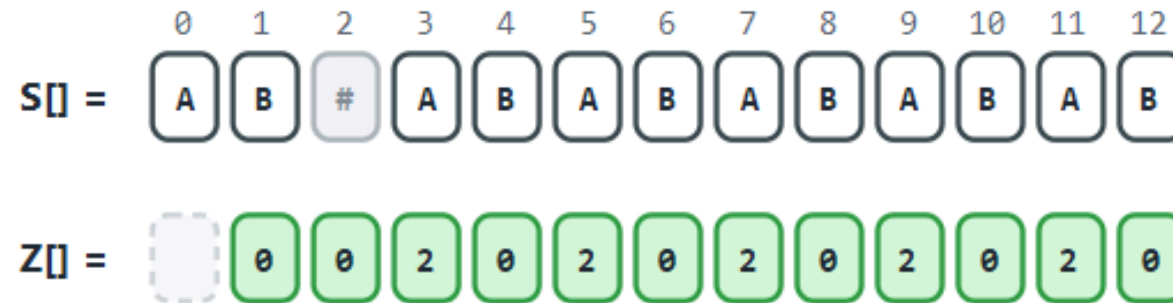
Şimdiye kadarki her algoritma kendi özel tablosunu kurdu. TEK bir kendisiyle-karşılaştırma tüm aramayı yanıtlayabilir mi?

Fikir: Z[i] ve birleşik bir dizgi

- $Z[i] : S[i..]$, S 'nin kendi başlangıcıyla ne kadar eşleşir?
- $S = \text{pattern} + \text{'\#'} + \text{text}$ olsun (başka yerde kullanılmayan bir ayraç)
- Metin kısmında $Z[i] \geq |\text{pattern}|$: bir oluşum
- Bir $[1, r)$ penceresi bilinen değerleri yeniden kullanır — toplam $O(n + m)$

Z algoritması, adım adım

pattern="AB" text="ABABABABAB" (birleşik: S = pattern + # + text)



Bitti: Z[] = [0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0]. text kısmında Z[i] >= 2 olan konumlar pattern'in oluşumlarıdır: 0, 2, 4, 6, 8. Toplam süre O(n + m).

Uç durum — pencere sürekli yeniden kullanılır

pattern="AAA" text="AAAAAAAAAA" (birleşik: S = pattern + # + text)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
S[] =	A	A	A	#	A	A	A	A	A	A	A	A	A	A
Z[] =		2	1	0	3	3	3	3	3	3	3	3	2	1

Bitti: Z[] = [0, 2, 1, 0, 3, 3, 3, 3, 3, 3, 3, 3, 2, 1]. text kısmında $Z[i] \geq 3$ olan konumlar pattern'in oluşumlarıdır: 0, 1, 2, 3, 4, 5, 6, 7. Toplam süre $O(n + m)$.

Kod — Z dizisi kurma

```
if (i < r)
    z[i] = min(r - i, z[i - 1]);
while (i + z[i] < n && s[z[i]] == s[i + z[i]])
    z[i]++;
if (i + z[i] > r) { l = i; r = i + z[i]; }
```

Karmaşıklık

- Tüm birleşik-dizgi Z dizisi için $O(n + m)$
- KMP'nin lps'siyle aynı amorti edilmiş argüman: pencere yalnız büyür
- Bir konumun bir oluşum işaretleyip işaretlemediğini kontrol etmek için $O(1)$
- Beş arama algoritmasının en basit olanı, kavramsal olarak

Sık yapılan hatalar

- Ayracı unutmak, ya da metinde geçen bir ayraç seçmek
- `Z[i] == m` yerine `Z[i] >= m` kullanmak
- `Z[0]` 'ı anlamlı saymak (geleneksel olarak kullanılmaz)

Mini soru

Z algoritması, KMP'nin lps'si gibi
AYRI bir tablo kurmaktan nasıl
kaçınır, tek bir cümlede?

Cevap

Kaçınmaz — `z[]` TABLONUN KENDİSİDİR, yalnız TÜM birleşik dizgiyle örtüşmeyi tanımlar.

12. Beş Arama Algoritmasını Karşılaştırmak

Karşılaştırma tablosu

Algoritma	En kötü durum	Tipik durum	Ekstra fikir
Saf	$O(n \cdot m)$	$O(n)$	Taban çizgisi
KMP	$O(n+m)$	$O(n+m)$	$lps[]$, i geri sarmaz
Rabin-Karp	$O(n \cdot m)$	$O(n+m)$ ort.	Kayan özet, doğrula
Boyer-Moore	$O(n \cdot m)$	$O(n/m)$	Sağdan sola, büyük atlama
Z algoritması	$O(n+m)$	$O(n+m)$	Tek kendisiyle-karşılaştırma dizisi

13. Yeni Bir Strateji: Dinamik Programlama

Başlangıç sorusu

İki dizgi ne kadar "farklı"? Yazım denetleyicileri, `git diff`, ve DNA araçları hepsi bu sorunun bir versiyonunu soruyor.

Saf kuvvet neden çok yavaş

- Özyinelemeli tanım: eşleştir, değiştir, sil, ya da ekle
- Doğru — ama neredeyse her adımda 3 çağrıya dallanır
- AYNİ alt problem üstel olarak sık yeniden hesaplanır
- Çakışan alt problemler: israf edilen iş düzeltilebilir

Fikir: her alt problemi bir kez çözün

- **Optimal alt yapı:** en iyi cevap, en iyi alt cevaplardan kurulur
- **Çakışan alt problemler:** aynısı birçok kez tekrar eder
- Her alt problemi TAM OLARAK BİR KEZ çözün, bir tabloda saklayın
- Bu dinamik programlamadır — bu hafta gerçekten yeni

Düzenleme uzaklığı: soru

- `a`'yı `b`'ye dönüştürmenin en az tek-karakterlik düzenlemesi
- Ekleme, silme, ya da değiştirme — her biri 1'e mal olur
- Ayrıca **Levenshtein uzaklığı** da denir (1965)
- `dp[i][j]` = `a`'nın ilk `i`'si ile `b`'nin ilk `j`'si arasındaki uzaklık

Yineleme

- Taban durumu: $dp[i][0] = i$, $dp[0][j] = j$
- Karakterler eşleşir: $dp[i][j] = dp[i-1][j-1]$ (ücretsiz)
- Aksi halde: $1 + \min(\text{köşegen}, \text{üst}, \text{sol})$
- Satır satır doldurun — her bağımlılık zaten hesaplanmış

Düzenleme uzaklığı, adım adım

	(boş)	S	I	T	T	I	N	G
(boş)	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
I	2	2	1	2	3	4	5	6
T	3	3	2	1	2	3	4	5
T	4	4	3	2	1	2	3	4
E	5	5	4	3	2	2	3	4
N	6	6	5	4	3	3	2	3

ops: substitute match match match substitute match insert

Bitti: $\text{edit_distance}(\text{"KITTEN"}, \text{"SITTING"}) = 3$. Bir en kısa düzenleme dizisi: substitute, match, match, match, substitute, match, insert. Tablo $O(n*m)$ sürede ve bellekte dolduruldu.

Uç durum — saf ekleme

	(boş)	C	A	T	E	R	P	I	L	L	A	R
(boş)	0	1	2	3	4	5	6	7	8	9	10	11
C	1	0	1	2	3	4	5	6	7	8	9	10
A	2	1	0	1	2	3	4	5	6	7	8	9
T	3	2	1	0	1	2	3	4	5	6	7	8

ops: match match match insert insert insert insert insert insert insert insert

Bitti: $\text{edit_distance}(\text{"CAT"}, \text{"CATERPILLAR"}) = 8$. Bir en kısa düzenleme dizisi: match, match, match, insert, insert, insert, insert, insert, insert, insert, insert. Tablo $O(n*m)$ sürede ve bellekte dolduruldu.

Kod — tabloyu doldurmak

```
if (a[i-1] == b[j-1])
    dp[i][j] = dp[i-1][j-1];
else {
    int best = min(dp[i-1][j-1],
                  min(dp[i-1][j], dp[i][j-1]));
    dp[i][j] = 1 + best;
}
```

Karmaşıklık

- $O(n*m)$ süre — her hücre bir kez, $O(1)$ 'de hesaplanır
- Tam tablo için $O(n*m)$ alan (geriye yürüme için gerekli)
- Saf özyinelemenin üstel süresine göre çarpıcı gelişme
- Yalnız-satır eniyilemesi alanı $O(m)$ 'ye indirir (yalnız uzunluk)

Sık yapılan hatalar

- Birer kayma: `dp[i][j]` , `a[i]` 'yı değil `a[i-1]` 'i karşılaştırır
- Taban durumlarının `i` ve `j` olduğunu, sıfır olmadığını unutmak
- Tabloyu tutarsız ya da yanlış bir sırada doldurmak

Mini soru

Dinamik programlama neden HEM optimal alt yapıya HEM çakışan alt problemlere ihtiyaç duyar?

Cevap

Alt yapı DOĞRU yapar. Çakışma ezberlemeyi DEĞERLİ yapar — çakışma olmadan, hiçbir şey yeniden hesaplanmaz.

14. En Uzun Ortak Alt Dizi

Başlangıç sorusu

"Ne kadar farklı" değil — "sırayla,
bitişik olmasa bile, NE AYNI kaldı"?

Alt dizi ile alt dizgi

- Alt dizgi: **bitişik** — "ACE", "ABCDE" 'nin bir alt dizgisi DEĞİL
- Alt dizi: sıra korunur, boşluklara izin verilir
- "ACE", "ABCDE" 'nin bir alt dizisi **DİR**
- LCS: HER İKİ dizgide de ortak en uzun dizi

Yineleme — neredeyse aynı tablo

- Taban durumu: $dp[i][0] = dp[0][j] = 0$
- Karakterler eşleşir: $dp[i][j] = dp[i-1][j-1] + 1$ (uzat)
- Aksi halde: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
- Uyuşmazlık için ceza yok — yalnız eşleşmeler bir şey ekler

LCS, adım adım

	(boş)	B	D	C	A	B	A
(boş)	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

LCS so far: BCBA

Bitti: LCS("ABCBADAB", "BDCABA") uzunluk 4, bir örnek: "BCBA". Tablo $O(n*m)$ sürede ve bellekte dolduruldu.

Uç durum — hiç paylaşılan harf yok

	(boş)	F	G	H	I	J
(boş)	0	0	0	0	0	0
A	0	0	0	0	0	0
B	0	0	0	0	0	0
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0

Bitti: LCS("ABCDE", "FGHIJ") uzunluk 0, bir örnek: "". Tablo $O(n*m)$ sürede ve bellekte dolduruldu.

Kod — yineleme

```
if (a[i-1] == b[j-1])
    dp[i][j] = dp[i-1][j-1] + 1;
else
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

Karmaşıklık

- $O(n*m)$ süre ve alan — düzenleme uzaklığıyla özdeş
- Aynı tablo şekli, aynı şekilde doldurulmuş
- Yalnız her hücredeki yineleme farklı
- Daha sonraki algoritma tasarımında yeniden tanıyacağınız bir şablon

Sık yapılan hatalar

- Alt diziyi alt dizgiyle karıştırmak (iki slayt önceye bakın)
- Yanlışlıkla düzenleme uzaklığının min+1 yinelemesini yeniden kullanmak
- Yalnız gerçek alt dizi isteniyorken uzunluğu bildirmek

Mini soru

LCS neden max kullanır, düzenleme uzaklığı min+1 kullanırken, ikisi de aynı tablo şeklini doldursa bile?

Cevap

Düzenleme uzaklığı bir MALİYET sayar (en ucuzu al). LCS bir UZUNLUK sayar (zaten en iyisini al).

Özet

- **Yapılar:** char dizisi + `'\0'`, büyüyen arabellekler, trie'ler, sıkıştırılmış trie'ler, son ek dizileri
- **Arama:** saf, KMP, Rabin-Karp, Boyer-Moore, Z algoritması
- **Bu hafta yeni:** dinamik programlama — düzenleme uzaklığı, LCS
- Aynı DP tablo şekli, iki farklı yineleme

Alıştırmalar ve kendini sinama

- Hafta notlarında 10 alıştırma — tabloları ve kodu elle izleyin
- Tam çözümlü 10 kendini sinama sorusu
- Seçicideki her "uç" örneği deneyin, yalnız "normal"i değil
- Her programı kendiniz çalıştırın — çıktıların hepsi gerçek, yakalanmış çıktı

İleriye bakış

- Hafta 13–14: proje haftaları 15–16'dan önce daha yeni malzeme
- DP tablo şekli (`dp[i][j]` , bir kez doldurulur, geriye yürümeyle okunur)
- ...yalnız dizgiler için değil, algoritma tasarımı boyunca tekrar tekrar karşınıza çıkar
- Bu haftanın tablo şeklini bir şablon olarak aklınızda tutun

Kaynaklar

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms*
- Knuth, Morris, Pratt (1977) · Boyer, Moore (1977) · Karp, Rabin (1987)
- Fredkin (1960) · Morrison (1968) · Levenshtein (1965)
- Sedgewick & Wayne — *Algorithms*, 4. baskı (Dizgiler bölümü)
- Tam liste ayrıntılarıyla hafta notlarında

Sorular?

CEN207 Veri Yapıları — Hafta 12

Gelecek hafta: Hafta 15–16'nın proje gösterimlerinden önce yeni malzemeye devam